

Hoisting (kéo lên)

Tự động đưa phần khai báo (declaration) của biến hoặc hàm lên đầu scope trước khi chạy code

Ví dụ với

- var:

```
console.log(a) // undefined
var a = 5
console.log(a) // 5
```

thực ra thì Javascript hiểu đoạn code này như sau:

```
var a;          // khai báo được hoisting lên đầu
console.log(a); // => undefined (chưa gán)
a = 5;          // gán giá trị sau
console.log(a); // => 5
```

- let, const: mặc dù JS vẫn biết là 2 từ nó tồn tại, nhưng từ nó không được *khởi tạo* giá trị (undefined) cho tới khi thực sự chạy đến dòng khai báo

```
console.log(b); // ❌ ReferenceError
let b = 10;

console.log(c); // ❌ ReferenceError
const c = 20;
```

- function hoisting

```
sayHello() // chạy được, ra Xin chào
```

```
function sayHello() {  
    console.log("Xin chào")  
}
```

- Nhưng với function expression thì không

```
sayHello()
```

```
var sayHello = function() {  
    console.log("Xin chào")  
}
```

Khai báo biến: var, let, const

```
if (true) {  
    var x = 10;  
}  
console.log(x); //  10 - vì var "thoát" khỏi block  
  
// còn let và const thì sẽ ReferenceError
```

```
// var khai báo lại biến được  
var x = 1  
var x = 2 // vẫn ok
```

=> var: cũ, dễ gây lỗi vì phạm vi rộng và hoisting

Data type

Javascript là dynamically typed nha, nó không có kiểu dữ liệu. Còn mấy cái mà

```
let name: string = "Gia Nguyên"  
let age: number = 21  
let isOnline: boolean = true
```

thì nó là Typescript rồi nha

Null với undefined

- undefined: Javascript tự gán. Biến chưa được gán giá trị hoặc function không trả về gì
- null: Dev gán thủ công. Giá trị rỗng có chủ đích để xóa hoặc làm rỗng

Những trường hợp xuất hiện undefined

```
// 1 Biến chưa gán  
let x;  
console.log(x); // undefined  
  
// 2 Hàm không return gì  
function test() {}  
console.log(test()); // undefined  
  
// 3 Truy cập thuộc tính không tồn tại  
let person = { name: "Alice" };  
console.log(person.age); // undefined  
  
// 4 Truy cập phần tử ngoài mảng
```

```
let arr = [1, 2, 3];  
console.log(arr[5]); // undefined
```

Kiểu dữ liệu của null và undefined

```
console.log(typeof null); // "object" 🤪 (bug lịch  
sử trong JS)  
console.log(typeof undefined); // "undefined"
```

So sánh == với ===

- == loose equality: nó sẽ tự ép kiểu (type coercion) trước khi so sánh
- === strict equality: so sánh cả KIỂU DỮ LIỆU và GIÁ TRỊ

```
// 🟡 Dùng ==  
console.log(5 == "5"); // ✅ true (vì "5" được ép  
thành số 5)  
console.log(0 == false); // ✅ true (vì false được ép  
thành 0)  
console.log(null == undefined); // ✅ true (đặc biệt  
trong JS)  
console.log(" " == 0); // ✅ true (chuỗi rỗng ép  
thành 0)
```

```
// 🔵 Dùng ===  
console.log(5 === "5"); // ❌ false (vì number ≠  
string)  
console.log(0 === false); // ❌ false (vì number ≠  
boolean)  
console.log(null === undefined); // ❌ false (kiểu khác  
nhau)  
console.log(" " === 0); // ❌ false
```

Câu điều kiện

```
if (a > 0) console.log("dương");  
else if (a < 0) console.log("âm");  
else console.log("0");
```

Mệnh đề switch case

```
switch(day) {  
  case "Mon": console.log("Thứ 2"); break;  
  default: console.log("Ngày khác");  
}
```

Loop

```
for (let i = 0; i < 5; i++) console.log(i);  
while (n > 0) { n--; }
```

Closure

```
function counter() {  
  let count = 0;  
  return () => ++count; // chú ý: chỗ này return về một  
  hàm nha  
}  
  
const c = counter();  
console.log(c()); // 1  
console.log(c()); // 2
```

```
const d = counter();
console.log(d()); // 1
console.log(d()); // 2
```

Closure (bao đóng): hàm counter trả về hàm con. Thì khi hàm cha bị xóa, hàm con vẫn còn tồn tại, mà hàm con lại sử dụng biến của hàm cha nên vẫn giữ lại

=> hàm con đóng gói luôn cả môi trường (scope) chứa nó

Array

```
let arr = [10, 20, 30, 40];
```

Phương thức	Mô tả	Ví dụ
<code>push()</code>	Thêm phần tử vào cuối mảng	<code>arr.push(50)</code> → <code>[10, 20, 30, 40, 50]</code>
<code>pop()</code>	Xóa phần tử cuối mảng	<code>arr.pop()</code> → <code>[10, 20, 30]</code>
<code>unshift()</code>	Thêm phần tử vào đầu mảng	<code>arr.unshift(0)</code> → <code>[0, 10, 20, 30, 40]</code>
<code>shift()</code>	Xóa phần tử đầu mảng	<code>arr.shift()</code> → <code>[20, 30, 40]</code>
<code>splice(start, count)</code>	Xóa/thêm phần tử bất kỳ	<code>arr.splice(2, 1)</code> → xóa 1 phần tử ở vị trí 2

Phương thức	Mô tả	Ví dụ
<code>forEach()</code>	Duyệt qua từng phần tử, không trả về mảng mới	<code>arr.forEach(x => console.log(x))</code>
<code>map()</code>	Tạo mảng mới sau khi biến đổi từng phần tử	<code>arr.map(x => x * 2)</code> → <code>[20, 40, 60, 80]</code>

Phương thức	Mô tả	Ví dụ
<code>filter()</code>	Tạo mảng mới với phần tử thỏa điều kiện	<code>arr.filter(x => x > 20)</code> → <code>[30, 40]</code>
<code>find()</code>	Trả về phần tử đầu tiên thỏa điều kiện	<code>arr.find(x => x > 20)</code> → <code>30</code>
<code>findIndex()</code>	Trả về chỉ số (index) của phần tử đầu tiên thỏa điều kiện	<code>arr.findIndex(x => x > 20)</code> → <code>2</code>
<code>reduce()</code>	Tính toán giá trị duy nhất từ mảng	<code>arr.reduce((a, b) => a + b, 0)</code> → <code>100</code>
<code>some()</code>	Trả về <code>true</code> nếu ít nhất 1 phần tử thỏa điều kiện	<code>arr.some(x => x > 30)</code> → <code>true</code>
<code>every()</code>	Trả về <code>true</code> nếu tất cả phần tử thỏa điều kiện	<code>arr.every(x => x > 5)</code> → <code>true</code>

Phương thức	Mô tả	Ví dụ
<code>sort()</code>	Sắp xếp (theo chuỗi mặc định)	<code>arr.sort()</code> hoặc <code>arr.sort((a,b)=>a-b)</code>
<code>reverse()</code>	Đảo ngược thứ tự	<code>arr.reverse()</code>
<code>join()</code>	Nối các phần tử thành chuỗi	<code>arr.join('-')</code> → <code>"10-20-30-40"</code>
<code>concat()</code>	Gộp mảng	<code>[1,2].concat([3,4])</code> → <code>[1,2,3,4]</code>
<code>slice(start, end)</code>	Cắt mảng con	<code>arr.slice(1,3)</code> → <code>[20,30]</code>

Phương thức	Mô tả	Ví dụ
<code>includes()</code>	Kiểm tra có chứa giá trị không	<code>arr.includes(20)</code> → <code>true</code>
<code>indexOf()</code>	Vị trí đầu tiên xuất hiện	<code>arr.indexOf(30)</code> → <code>2</code>
<code>lastIndexOf()</code>	Vị trí cuối cùng xuất hiện	<code>[1,2,1].lastIndexOf(1)</code> → <code>2</code>

Phương thức	Mô tả	Ví dụ
<code>Array.isArray()</code>	Kiểm tra có phải mảng không	<code>Array.isArray(arr) → true</code>

Object

```
let person = {
  name: "Alice",
  age: 25,
  job: "Developer"
};
```

Cách	Mô tả	Ví dụ
<code>object.key</code>	Truy cập giá trị	<code>person.name → "Alice"</code>
<code>object["key"]</code>	Truy cập bằng chuỗi	<code>person["age"] → 25</code>
Gán giá trị mới	Cập nhật / thêm key	<code>person.city = "Hanoi"</code>

Phương thức	Mô tả	Ví dụ
<code>Object.keys(obj)</code>	Mảng chứa tất cả key	<code>["name", "age", "job"]</code>
<code>Object.values(obj)</code>	Mảng chứa tất cả value	<code>["Alice", 25, "Developer"]</code>
<code>Object.entries(obj)</code>	Mảng chứa cặp [key, value]	<code>[["name", "Alice"], ["age", 25]]</code>
Duyệt bằng <code>for...in</code>	Lặp qua các key	<code>for (let key in person) { console.log(key, person[key]); }</code>

Spread operator


```
Array.from({ length: 3 }, (_, i) => i)
// → [0, 1, 2]
```

```
let n = 255;
```

```
console.log(n.toString(2)); // "11111111" → hệ nhị  
phân (base 2)  
console.log(n.toString(8)); // "377" → hệ bát  
phân (base 8)  
console.log(n.toString(10)); // "255" → hệ thập  
phân (base 10)  
console.log(n.toString(16)); // "ff" → hệ thập  
lục phân (base 16)  
console.log(n.toString(36)); // "73" → hệ 36 (sử  
dụng cả chữ cái)
```